Git & GitHub Usage Guidelines for Software Development

Branching Strategy

Our repositories use the following branches:

- **main**: Always contains the latest released production version.
- **prod**: Used for production/release deployment.
- **stag**: Used for testing and staging.
- **dev**: Used for active development.

Workflow

1. **Clone the `dev` Branch Only**
    - Always start by cloning the repository from the `dev` branch.

      ```
      git clone -b dev git@github.com:wahsandaruwan/your-repo.git
      ```

    - Do **NOT** commit directly to any branch except via pull requests.
2. **Feature/Bugfix Branches**
    - Create a feature or bugfix branch from `dev` :
        - Feature: `feature/<short-description>`
        - Bugfix: `bugfix/<short-description>`
    - Example:

      ```
      git checkout dev
      git pull origin dev
      git checkout -b feature/user-auth
      ```

    - Push your branch to GitHub:

      ```
      git push -u origin feature/user-auth
      ```

3. **Making Changes**

   - Commit your changes to your feature or bugfix branch.
   - Push your changes to GitHub:

```
git add .
git commit -m "Add user authentication"
git push
```

4. **Create Pull Request (PR) – Always Remotely**

   - **Pull Requests must be created via the GitHub web interface, NOT locally with CLI tools.**
   - Go to the repository on GitHub.
   - Click "Compare & pull request" for your branch.
   - Ensure the PR is **from your feature/bugfix branch to** `dev`.
   - Assign at least **2 reviewers** from the team.
   - Wait for their approval before merging.

5. **Merge and Cleanup**

   - After PR approval, merge your branch into `dev` using the GitHub web interface.
   - Delete your feature/bugfix branch both remotely and locally:

```
git branch -d feature/user-auth
git push origin --delete feature/user-auth
```

6. **Testing**

   - If testing is required, merge the latest `dev` into `stag`:

```
git checkout stag
git merge dev
git push origin stag
```

   - Use the `stag` branch for QA, UAT, or other testing.

7. **Production Release**

   - When ready for production:
     - Merge `stag` into `prod`:

```
git checkout prod
git merge stag
git push origin prod
```

- Also update `main` to the latest production version:

```
git checkout main
git merge prod
git push origin main
```

GitHub Access (Private Repos) & SSH Setup

All our repositories are private. Team members must set up SSH access to GitHub.

SSH Setup Steps for Windows 11

1. **Install Git for Windows**
   - Download and install from: https://git-scm.com/download/win
2. **Check for Existing SSH Keys**
   - Open **Git Bash** and run:

```
ls -al ~/.ssh
```

   - Look for files named `id_rsa` and `id_rsa.pub` (or similar).
3. **Generate New SSH Key (if none exists)**
   - In **Git Bash**, run:

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

   - Press Enter to accept default location and set a passphrase (optional).
4. **Add SSH Key to SSH Agent**
   - Start the agent:

```
eval "$(ssh-agent -s)"
```

   - Add your key:

```
ssh-add ~/.ssh/id_ed25519
```

5. **Add Public Key to GitHub**
    - Copy your public key:

      ```
      cat ~/.ssh/id_ed25519.pub
      ```

    - Copy the displayed key.
    - Go to GitHub SSH Settings, click **New SSH key**, paste your key, and save.
6. **Test Connection**
    - Run:

      ```
      ssh -T git@github.com
      ```

    - You should see a success message.
7. **Clone Repository via SSH**
    - Use the SSH URL (not HTTPS):

      ```
      git clone -b dev git@github.com:wahsandaruwan/your-repo.git
      ```

General Best Practices

- **Commit Often**: Make small, frequent commits with clear messages.
- **Review Thoroughly**: Participate in code reviews and provide constructive feedback.
- **Keep Branches Clean**: Delete feature/bugfix branches after merging.
- **Never Commit Secrets**: Do not commit passwords, API keys, or sensitive data.

Example Branch Lifecycle

```
(dev) → [feature/bugfix branch] → PR (created remotely) → (dev) → (stag) → (prod) → (main)
```

WAHSAND